

Chapter 8

Wrapping Up

From prototype to production,
and where to go next

Learning Objectives

- Approach a real ML problem **end to end**
- Understand the human role and the **production** realities of ML systems
- Build your own scikit-learn-compatible **estimator**
- Know where to go next: **theory, frameworks, and other kinds of learning**

Part 1

ML in the Real World

Approaching a Machine Learning Problem

Don't start by running your favorite algorithm. Step back and ask:

1. **What question** am I answering? Exploratory, or a specific goal?
2. How will I **define and measure success**?
3. Do I have the **right data** to evaluate it?
4. What is the **business (or research) impact** of a good solution?

The algorithm is only a **small part** of a larger data-analysis process.

Is the Problem Worth Solving?

A useful thought experiment: **"What if I built the perfect model?"**

- Perfect fraud detection that saves **\$100/month** → not worth the effort
- One that saves **tens of thousands/month** → worth exploring
- Model building sits in a **feedback loop**: collect → clean → model → analyze
- **Analyzing mistakes** reveals what data to add or how to reframe the task
- Collecting better data often beats **endless grid searches** for accuracy

Humans in the Loop

ML rarely replaces human judgment — it **augments** it.

- Some decisions must be **immediate** (pedestrian detection in a self-driving car)
- Others can route **uncertain** cases to a human for confirmation
- **Medical** applications may need precision beyond what a model alone can reach

*Automating even 90%, 50%, or 10% of decisions can cut cost or response time — handle the **simple cases**, reroute the **complicated** ones to people.*

Part 2

From Prototype to Production

From Prototype to Production

A notebook prototype is **not** a production system.

- Python & scikit-learn *are* used in production — even at big banks and social networks
- But analytics teams (Python / R) and production teams (**Go, Scala, C++, Java**) often differ
- A common path: **reimplement** the chosen solution in a high-performance language
- Production values **reliability, latency, and memory** — keep the pipeline **simple**

See "Machine Learning: The High-Interest Credit Card of Technical Debt" (Google).

Testing Production Systems

Offline

- Evaluate on **held-out** historical data
- What we've done all book
- **A/B testing**: show algorithm A to some users, B to others, compare metrics
- More advanced: **bandit algorithms** for adaptive online testing
- Live behavior can differ drastically from offline scores → always plan to monitor

Online (live)

- Test with **real users**
- Catches "in the wild" effects

Building Your Own Estimator

Implement the scikit-learn interface → it plugs into **pipelines** and **grid search**.

```
from sklearn.base import BaseEstimator, TransformerMixin

class MyTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, first_parameter=1, second_parameter=2):
        self.first_parameter = first_parameter
        self.second_parameter = second_parameter
    def fit(self, X, y=None):          # accept y even if unused
        return self
    def transform(self, X):
        return X + 1
```

For a model, inherit `ClassifierMixin` / `RegressorMixin` and implement `predict`.

Part 3

Where to Go From Here

Deepen the Theory

This book built **intuition**; theory makes you a better practitioner.

- *The Elements of Statistical Learning* — Hastie, Tibshirani, Friedman
- *Machine Learning: An Algorithmic Perspective* — Marsland (with Python code)
- *Pattern Recognition and Machine Learning* — Bishop (probabilistic)
- *Machine Learning: A Probabilistic Perspective* — Murphy (comprehensive)

Other Frameworks & Packages

scikit-learn isn't the only tool — pick what fits the job.

Tool	Good for
statsmodels	statistical modeling & inference
R	statistics, specialized models, visualization
vowpal wabbit (vw)	large / streaming data (C++)
spark + MLlib	distributed learning on a cluster

Other Kinds of Learning

Beyond classification, regression, and clustering:

- **Ranking** — return an ordered list for a query (how search engines work)
- **Recommender systems** — "People You May Know", the **Netflix prize** (\$1M)
- **Time series** — forecasting (e.g. stock prices), with its own literature

Seek out books, papers, and communities for the paradigm that fits your problem.

Probabilistic Modeling & Programming

Many real problems have **structure** you can encode with probability theory.

- Example: indoor navigation fusing **GPS + accelerometer + compass** with a known map
- A black-box model throws away what you *already know* about the world
- **Probabilistic programming** languages express such models elegantly
- Tools: **PyMC** (Python) and **Stan** (multi-language)

Neural Networks & Deep Learning

We only touched neural nets (Chapters 2 & 7) — the field moves **weekly**.

- Recent breakthroughs: **AlphaGo**, speech understanding, near-instant translation
- Any "state of the art" reference dates quickly
- Deepen with the standard text: *Deep Learning* — Goodfellow, Bengio, Courville

Scaling to Larger Datasets

This book assumed data fits in **RAM**. When it doesn't, two strategies:

- **Out-of-core learning** — stream the data in **chunks**, update the model per chunk, on a single machine (supported by some scikit-learn models)
- **Parallelization over a cluster** — distribute data across machines (**spark + MLlib**, or `vw`)

Most real datasets are small enough for one machine — reach for these only when truly needed.

Honing Your Skills

Only **practice** turns knowledge into expertise.

- **Competitions** (Kaggle) — a given task and dataset, teams compete on predictions
- **OpenML** — 20,000+ datasets with 50,000+ associated tasks
- ⚠ Competitions hand you a **fixed metric and preprocessed data**
- In the real world, **defining the problem and collecting data** matter more than the last 1% of accuracy

Course Conclusion

- ML **extracts knowledge from data** — and is easy to apply in practice
- Master the workflow: **frame** → **represent** → **model** → **evaluate** → **iterate**
- The **representation** of your data often matters more than the algorithm
- Keep humans, production realities, and the **business goal** in view

Keep digging into the data, and don't lose sight of the larger picture.

Thank you!

Introduction to Machine Learning

Based on *Introduction to Machine Learning with Python*
Andreas C. Müller & Sarah Guido